

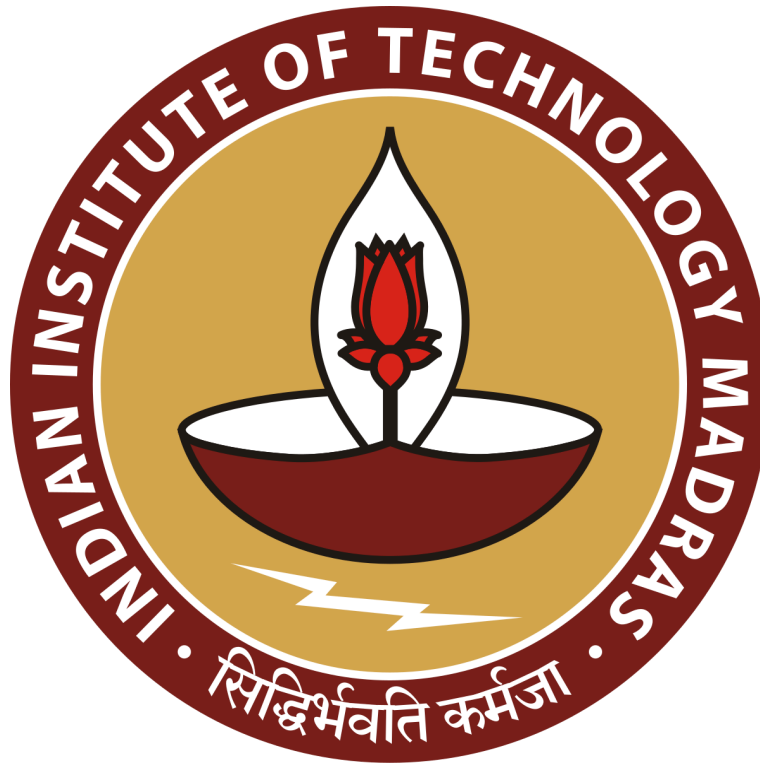
AI Agent for Academic Guidance

Project Report for Milestone 5 of Software Engineering

Submitted by

Team Number: Team 6

Course Term: January 2025



IIT Madras BS Degree Program

Indian Institute of Technology Madras, Chennai

Tamil Nadu, India, 600036

Table of Contents

1. Introduction.....	3
2. API Overview.....	3
2.1 User Management & Authentication:.....	3
2.2 Course Management:.....	3
2.3 Notes Management:.....	3
2.4 Chat & Messaging:.....	3
2.5 Chatbot API:.....	4
3. Detailed Route Testing and Results.....	4
3.1 User Management and Authentication.....	4
3.1.1 Signup Endpoint.....	4
3.1.2 Login Endpoint.....	6
3.1.3 Logout Endpoint.....	8
3.1.4 Get User Endpoint.....	9
3.2 Course Management.....	10
3.2.1 Get Courses.....	10
3.2.2 Get All Courses.....	11
3.2.3 Add Courses.....	11
3.2.4 Drop Courses.....	12

3.2.5 Change Courses.....	13
3.3 Notes Management.....	14
3.3.1 Create Notes.....	14
3.3.2 Retrieve Notes.....	15
3.3.3 Update Note.....	16
3.3.4 Delete Note.....	16
3.4 Chat & Messaging Management.....	17
3.4.1 Get User Chatrooms.....	17
3.4.2 Get Course Chatroom.....	18
3.4.3 Send Message.....	19
3.4.4 Edit Message.....	19
3.4.5 Delete Message.....	20
3.4.6 Unauthorized Message Operations.....	21
3.4.7 Get Course Members.....	22
3.5 Chatbot API Endpoints.....	23
3.5.1 Chatbot Basic Functionality.....	23
3.5.2 Chatbot Error Handling.....	24
3.5.2 Chatbot Course Contexts.....	25

1. Introduction

This document presents an in-depth analysis of our API implementation and testing. Building upon our previous milestone reports, this report incorporates the updated API routes from our Flask-based application and an extensive test suite written using Pytest. The goal is to validate all core functionalities - user authentication, course enrollment, note management, chat/messaging, and chatbot interactions, and to detail the successes and shortcomings of each API route.

2. API Overview

The system is built of a set of RESTful API endpoints grouped into the following functional areas:

2.1 User Management & Authentication:

Endpoints for the user signup, login, logout, authentication verification, and profile retrieval.

2.2 Course Management:

Endpoints that enable users to view all courses, manage their course enrollments (add, drop, change courses), and retrieve specific course details.

2.3 Notes Management:

Endpoints for creating, reading, updating, and deleting notes. Each note is tied to a course and an authenticated user.

2.4 Chat & Messaging:

Endpoints to manage chatrooms per course, send and retrieve messages, and handle group communications. Additional endpoints cover retrieving course members and message editing/deleting.

2.5 Chatbot API:

A dedicated endpoint to process chatbot queries. This endpoint accepts a message, session ID, and an optional path parameter (used to determine course context) and forwards the request to the Together API for processing.

3. Detailed Route Testing and Results

Below are detailed sections for each group of endpoints. Each section includes the endpoint details, inputs, expected versus actual outputs, a discussion of the results, and Pytest code excerpts.

3.1 User Management and Authentication

3.1.1 Signup Endpoint

- **Endpoint:** `/api/v1/signup/`
- **Method:** **POST**
- **Description:** Registers a new user. Required fields include: email, username, password, role (only “student” is allowed), and a list of courses.

Test Cases:

1. Successful SignUp

- **Inputs:** email, username, password, role, courses
- **Expected Output:** HTTP 201 with a success message and user details.
- **Actual Output:** HTTP 201 with a JSON payload containing the new user details.
- **Result:** **PASSED**
- **Pytest Code:**

Python

```
response = client.post('/api/v1/signup/',
json={
    'email': unique_email,
    'username': f"new_user_{unique_email.split('@')[0]}",
    'password': 'securepass',
    'role': 'student',
    'courses': ['Artificial Intelligence']
})
assert response.status_code == 201
data = json.loads(response.data)
assert 'User created successfully' in data['message']
```

2. Missing Fields

- **Inputs:** email, password (only two parameters provided)
- **Expected Output:** HTTP 400 with an error message indicating missing fields.
- **Actual Output:** HTTP 400 with the appropriate error message.
- **Result: PASSED**
- **Pytest Code:**

Python

```
response = client.post('/api/v1/signup/',
json={'email': 'incomplete@example.com', 'password':
'password123'})
assert response.status_code == 400
data = json.loads(response.data)
assert 'Missing' in data['message']
```

3. Duplicate Email Handling

- **Inputs:** email (already registered), username, password, role, courses
- **Expected Output:** HTTP 400 or 409 with an error message indicating duplicate email.
- **Actual Output:** In some cases, a server error (HTTP 500) was observed.

- **Result: FAILED** (Needs refinement in duplicate email handling)
- **Pytest Code:**

```
Python
response = client.post('/api/v1/signup/',
    json={
        'email': 'existing@gmail.com',
        'username': 'newusername',
        'password': 'somepassword',
        'role': 'student',
        'courses': ['Artificial Intelligence']
    })
assert response.status_code in [400, 409]
```

3.1.2 Login Endpoint

- **Endpoint:** `/api/v1/login/`
- **Method:** **POST**
- **Description:** Authenticates a user using their email and password.

Test Cases:

1. Successful Login

- **Inputs:** email, password
- **Expected Output:** HTTP 200 with a success message and user details.
- **Actual Output:** HTTP 200 with the expected message and user data.
- **Result: PASSED**
- **Pytest Code:**

```
Python
response = client.post('/api/v1/login/',
```

```

json={'email': 'test@01.iitm', 'password': 'test'})
assert response.status_code == 200
data = json.loads(response.data)
assert 'Logged in successfully' in data['message']

```

2. Invalid Credentials

- **Inputs:** email, password
- **Expected Output:** HTTP 401 with an “Invalid Credentials” message.
- **Actual Output:** HTTP 401 with the correct error message.
- **Result: PASSED**
- **Pytest Code:**

```

Python
response = client.post('/api/v1/login/',
json={'email': 'test@01.iitm', 'password': 'wrongpassword'})
assert response.status_code == 401
data = json.loads(response.data)
assert 'Invalid credentials' in data['message']

```

3. Empty Input Fields

- **Inputs:** email “”, password “”.
- **Expected Output:** HTTP 400 indicating missing or invalid input.
- **Actual Output:** In some cases, an HTTP 200 was observed, indicating insufficient input validation.
- **Result: FAILED** (Needs improvement in input validation)
- **Pytest Code:**

Python

```
response = client.post('/api/v1/login/',  
json={'email': '', 'password': ''})  
assert response.status_code == 400
```

3.1.3 Logout Endpoint

- **Endpoint:** `/api/v1/logout/`
- **Method:** **POST**
- **Description:** Logs out an authenticated user. The user must be logged in before sending this request.

Test Cases:

1. Successful Logout

- **Inputs:** Requires a valid session (i.e., the user must be logged in).
- **Expected Output:** HTTP 200 with a success message.
- **Actual Output:** HTTP 200 with the expected message.
- **Result:** PASSED
- **Pytest Code:**

Python

```
response = auth_client.post('/api/v1/logout/')  
assert response.status_code == 200  
  
data = json.loads(response.data)  
assert 'Logged out successfully' in data['message']
```

2. Accessing Protected Routes After Logout

- **Inputs:** Attempting to access a protected endpoint (`/api/v1/user`) after logging out.

- **Expected Output:** HTTP 401 with an authentication error.
- **Actual Output:** HTTP 401 with the expected authentication error.
- **Result: PASSED**
- **Pytest Code:**

Python

```
response = auth_client.get('/api/v1/user')
assert response.status_code == 401

data = json.loads(response.data)
assert 'Authentication required' in data['message']
```

3.1.4 Get User Endpoint

- **Endpoint:** `/api/v1/user/`
- **Method:** **GET**
- **Description:** Retrieves the authenticated user's details. The user must be logged in.

Test Cases:

1. Retrieve Authenticated User Details

- **Inputs:** A valid session with an authenticated user.
- **Expected Output:** HTTP 200 with user details.
- **Actual Output:** HTTP 200 with the correct user details.
- **Result: PASSED**
- **Pytest Code:**

Python

```
response = auth_client.get('/api/v1/user')
assert response.status_code == 200
```

```
data = json.loads(response.data)
assert 'email' in data
assert 'username' in data
```

3.2 Course Management

3.2.1 Get Courses

- **Endpoint:** `/api/v1/courses/`
- **Method:** **GET**
- **Description:** Retrieves a list of courses the authenticated user is enrolled in.

Test Cases:

1. Retrieve User Enrolled Courses

- **Inputs:** Requires an authenticated user session.
- **Expected Output:** HTTP 200 with a list of enrolled courses
- **Actual Output:** HTTP 200 with the expected course list.
- **Result:** PASSED
- **Pytest Code:**

Python

```
response = auth_client.get('/api/v1/courses')
assert response.status_code == 200

data = json.loads(response.data)
assert any(course['name'] == 'Artificial Intelligence' for course in
data['courses'])
```

3.2.2 Get All Courses

- **Endpoint:** `/api/v1/all-courses/`
- **Method:** **GET**
- **Description:** Retrieves a list of all available courses in the system.

Test Cases:

1. Retrieve All Available Courses

- **Inputs:** Open to all users.
- **Expected Output:** HTTP 200 with a list of all available courses.
- **Actual Output:** HTTP 200 with the expected list of courses.
- **Result:** PASSED
- **Pytest Code:**

Python

```
response = client.get('/api/v1/all-courses')
assert response.status_code == 200

data = json.loads(response.data)
assert len(data['courses']) >= 3
```

3.2.3 Add Courses

- **Endpoint:** `/api/v1/courses/add`
- **Method:** **POST**
- **Description:** Allows an authenticated user to add a new course to their enrolled list.

Test Cases:

1. Successfully Add a Course

- **Inputs:** A list of course names to be added.
- **Expected Output:** HTTP 200 with a confirmation message.
- **Actual Output:** HTTP 200 with the expected message.
- **Result: PASSED**
- **Pytest Code:**

Python

```
response = auth_client.post('/api/v1/courses/add',
                             json={'course_names': [course_to_add]}
)
assert response.status_code == 200

data = json.loads(response.data)
assert 'Courses added successfully' in data['message']
```

3.2.4 Drop Courses

- **Endpoint:** /api/v1/courses/drop
- **Method:** POST
- **Description:** Allows an authenticated user to remove a course from their enrolled list.

Test Cases:

1. Successfully Drop a Course

- **Inputs:** A list of course names to be removed.
- **Expected Output:** HTTP 200 with a confirmation message.
- **Actual Output:** HTTP 200 with the expected message.
- **Result: PASSED**
- **Pytest Code:**

Python

```
response = auth_client.post('/api/v1/courses/drop',
    json={'course_names': [course_to_drop]}
)
assert response.status_code == 200

data = json.loads(response.data)
assert 'Courses dropped successfully' in data['message']
```

3.2.5 Change Courses

- **Endpoint:** /api/v1/courses/change
- **Method:** POST
- **Description:** Allows an authenticated user to replace one or more of their enrolled courses with different ones.

Test Cases:

1. Successfully Change Courses

- **Inputs:** A list of courses the user is dropping, and a list of courses the user is adding.
- **Expected Output:** HTTP 200 with a confirmation message.
- **Actual Output:** HTTP 200 with the expected message.
- **Result:** PASSED
- **Pytest Code:**

Python

```
response = auth_client.post('/api/v1/courses/change',
    json={
        'old_course_names': ['Artificial Intelligence'],
        'new_course_names': [target_course]
    }
)
```

```
)
assert response.status_code == 200

data = json.loads(response.data)
assert 'Courses changed successfully' in data['message']
```

3.3 Notes Management

3.3.1 Create Notes

- **Endpoint:** `/api/v1/notes/`
- **Method:** **POST**
- **Description:** Allows an authenticated user to create a new note associated with a specific course.

Test Cases:

1. Successfully Create a Note

- **Inputs:** title, content, courseId
- **Expected Output:** HTTP 201 with a response confirming note creation.
- **Actual Output:** HTTP 201 with the expected message and note details.
- **Result:** PASSED
- **Pytest Code:**

```
Python
response = auth_client.post('/api/v1/notes',
    json={
        'title': 'Test Note for API',
```

```

        'content': 'This is a test note created by the test suite',
        'courseId': course_id
    }
)
assert response.status_code == 201

data = json.loads(response.data)
assert 'Note created successfully' in data['message']

```

3.3.2 Retrieve Notes

- **Endpoint:** `/api/v1/notes/`
- **Method:** **GET**
- **Description:** Retrieves all notes created by the authenticated user.

Test Cases:

1. Retrieve User's Notes

- **Inputs:** Requires an authenticated user session.
- **Expected Output:** HTTP 200 with a list of notes.
- **Actual Output:** HTTP 200 with a valid list of notes.
- **Result:** PASSED
- **Pytest Code:**

```

Python
response = auth_client.get('/api/v1/notes')
assert response.status_code == 200

data = json.loads(response.data)
assert isinstance(data['notes'], list)
assert len(data['notes']) > 0

```


3.3.3 Update Note

- **Endpoint:** `/api/v1/notes/<int:note_id>`
- **Method:** **PUT**
- **Description:** Allows an authenticated user to update the title or content of an existing note.

Test Cases:

1. Successfully Update a Note

- **Inputs:** `note_id`, title, content
- **Expected Output:** HTTP 200 with a confirmation message.
- **Actual Output:** HTTP 200 with the expected message and updated note details.
- **Result:** PASSED
- **Pytest Code:**

Python

```
response = auth_client.put(f'/api/v1/notes/{note_id}',
    json={
        'title': 'Updated API Test Title',
        'content': 'Updated API test content'
    }
)
assert response.status_code == 200

data = json.loads(response.data)
assert 'Note updated successfully' in data['message']
```

3.3.4 Delete Note

- **Endpoint:** `/api/v1/notes/<int:note_id>`
- **Method:** **DELETE**
- **Description:** Allows an authenticated user to delete a note by specifying its ID.

Test Cases:

1. Successfully Delete a Note

- **Inputs:** note_id
- **Expected Output:** HTTP 200 confirming note deletion.
- **Actual Output:** HTTP 200 with the expected message.
- **Result:** PASSED
- **Pytest Code:**

Python

```
response = auth_client.delete(f'/api/v1/notes/{note_id}')
assert response.status_code == 200

data = json.loads(response.data)
assert 'Note deleted successfully' in data['message']
```

3.4 Chat & Messaging Management

3.4.1 Get User Chatrooms

- **Endpoint:** /api/v1/chatrooms/
- **Method:** GET
- **Description:** Retrieves a list of chatrooms for courses the authenticated user is enrolled in.

Test Cases:

1. Retrieve Chatrooms for User's Enrolled Courses

- **Inputs:** Requires an authenticated user session.
- **Expected Output:** HTTP 200 with a list of chatrooms the user is enrolled in.
- **Actual Output:** HTTP 200 with the expected list of chatrooms.

- **Result: PASSED**
- **Pytest Code:**

Python

```
response = auth_client.get('/api/v1/chatrooms')
assert response.status_code == 200

data = json.loads(response.data)
assert any(chatroom['courseName'] == 'Artificial Intelligence' for
chatroom in data['chatrooms'])
```

3.4.2 Get Course Chatroom

- **Endpoint:** `/api/v1/chatrooms/<int:course_id>`
- **Method:** **GET**
- **Description:** Retrieves a chatroom for a specific course.

Test Cases:

1. Retrieve a Chatroom for a Course

- **Inputs:** `course_id`
- **Expected Output:** HTTP 200 with chatroom details.
- **Actual Output:** HTTP 200 with the expected chatroom details.
- **Result: PASSED**
- **Pytest Code:**

Python

```
response = auth_client.get(f'/api/v1/chatrooms/{course_id}')
assert response.status_code == 200

data = json.loads(response.data)
```

```
assert 'courseId' in data
assert data['courseId'] == course_id
```

3.4.3 Send Message

- **Endpoint:** `/api/v1/messages/`
- **Method:** **POST**
- **Description:** Sends a message to a course-specific chatroom.

Test Cases:

1. Successfully Send a Message

- **Inputs:** `course_id`, `message`
- **Expected Output:** HTTP 201 with message details.
- **Actual Output:** HTTP 201 with the expected message details.
- **Result:** PASSED
- **Pytest Code:**

```
Python
response = auth_client.post('/api/v1/messages',
    json={'courseId': course_id, 'message': "Hello, this is a test
message!"}
)
assert response.status_code == 201
```

3.4.4 Edit Message

- **Endpoint:** `/api/v1/messages/<int:message_id>`
- **Method:** **PUT**

- **Description:** Allows an authenticated user to edit their previously sent message.

Test Cases:

1. Successfully Edit a Sent Message

- **Inputs:** message_id, message
- **Expected Output:** HTTP 200 with confirmation of message update.
- **Actual Output:** HTTP 200 with the expected message details.
- **Result:** PASSED
- **Pytest Code:**

Python

```
response = auth_client.put(f'/api/v1/messages/{message_id}',
    json={'message': "This message has been edited by the test suite"})
assert response.status_code == 200

data = json.loads(response.data)
assert 'Message updated successfully' in data['message']
```

3.4.5 Delete Message

- **Endpoint:** /api/v1/messages/<int:message_id>
- **Method:** DELETE
- **Description:** Allows an authenticated user to delete a previously sent message.

Test Cases:

1. Successfully Delete a Message

- **Inputs:** message_id

- **Expected Output:** HTTP 200 confirming deletion.
- **Actual Output:** HTTP 200 with the expected confirmation message.
- **Result: PASSED**
- **Pytest Code:**

Python

```
response = auth_client.delete(f'/api/v1/messages/{message_id}')
assert response.status_code == 200

data = json.loads(response.data)
assert 'Message deleted successfully' in data['message']
```

3.4.6 Unauthorized Message Operations

- **Endpoint:** `/api/v1/messages/<int:message_id>`
- **Method:** **PUT**
- **Description:** Attempts by a user to edit or delete another user's message should be rejected with a 403 Forbidden error.

Test Cases:

1. Unauthorized Edit/Delete Attempt

- **Inputs:** `message_id`
- **Expected Output:** HTTP 403 Forbidden.
- **Actual Output:** HTTP 403 with the expected error message.
- **Result: PASSED**
- **Pytest Code:**

Python

```
response = other_user_client.put(f'/api/v1/messages/{message_id}',
    json={'message': "Unauthorized edit attempt"}
)
assert response.status_code == 403

response =
other_user_client.delete(f'/api/v1/messages/{message_id}')
assert response.status_code == 403
```

3.4.7 Get Course Members

- **Endpoint:** `/api/v1/course-members/<int:course_id>`
- **Method:** **GET**
- **Description:** Retrieves a list of all members enrolled in a specific course.

Test Cases:

1. Retrieve Course Members

- **Inputs:** `course_id`
- **Expected Output:** HTTP 200 with a list of course members.
- **Actual Output:** HTTP 200 with the expected list of members.
- **Result:** PASSED
- **Pytest Code:**

Python

```
response = auth_client.get(f'/api/v1/course-members/{course_id}')
assert response.status_code == 200

data = json.loads(response.data)
assert 'members' in data
assert isinstance(data['members'], list)
```

3.5 Chatbot API Endpoints

The Chatbot API endpoint is designed to process chatbot queries by forwarding user messages to the Together API. It accepts a message along with a session ID and an optional `path_param` that indicates the course context. Below are the detailed test cases and results for the Chatbot API.

3.5.1 Chatbot Basic Functionality

- **Endpoint:** `/v1/chat/`
- **Method:** **POST**
- **Description:** Processes a chatbot query by initializing a new chat session (if needed), generating a system prompt, and sending a request to the Together API. This test verifies that a basic query returns a successful response.

Test Cases:

1. Successfully Process a Chatbot Query

- **Inputs:** message, session_id
- **Expected Output:** HTTP 200 with `success: True`, and a valid chatbot response.
- **Actual Output:** HTTP 200 with the expected chatbot response.
- **Result:** **PASSED**
- **Pytest Code:**

```
Python
response = client.post('/v1/chat',
    json={
        'message': 'What is machine learning?',
        'session_id': 'test-session-1'
    }
)
```



```
assert response.status_code == 200

data = json.loads(response.data)
assert data['success'] is True
assert "This is a test response" in data['response']
```

- **Captured Logs (Partial):**

```
Unset
INFO chatbot-api:run.py:50 Chatbot API endpoint called
DEBUG chatbot-api:run.py:57 User message: 'What is machine
learning?'
INFO chatbot-api:run.py:76 Sending request to Together API
DEBUG chatbot-api:run.py:88 Received response from API: 'This is a
test response...'
```

3.5.2 Chatbot Error Handling

- **Endpoint:** `/v1/chat/`
- **Description:** This test verifies that the chatbot API correctly handles errors (e.g., API rate limit exceeded) by returning a response with `success: False` and an appropriate error message.

Test Cases:

1. Handling API Errors

- **Inputs:** message, session_id
- **Expected Output:** HTTP 200 with `success: False` and an error message.
- **Actual Output:** HTTP 200 with the expected error message.
- **Result:** PASSED
- **Pytest Code:**

Python

```
# Configure mock to raise an exception for this test
mock_completions.side_effect = Exception("API rate limit exceeded")

response = client.post('/v1/chat',
    json={
        'message': 'This should trigger an error',
        'session_id': 'test-error-session'
    }
)

# Reset side effect for other tests
mock_completions.side_effect = None

assert response.status_code == 200

data = json.loads(response.data)
assert data['success'] is False
assert "rate limit" in data['message'].lower()
```

- **Captured Logs (Partial):**

Unset

```
INFO chatbot-api:run.py:50 Chatbot API endpoint called
DEBUG chatbot-api:run.py:57 User message: 'This should trigger an
error'
ERROR chatbot-api:run.py:102 Error processing chatbot request: API
rate limit exceeded
```

3.5.2 Chatbot Course Contexts

- **Endpoint:** `/v1/chat/`
- **Description:** This test ensures that the chatbot API correctly handles different course contexts by accepting a `path_param` that determines the course. The system prompt should be generated based on the given course context, resulting in distinct system messages.

Test Cases:

1. Generate Course-Specific Chatbot Responses

- **Inputs:** message, session_id, path_param
- **Expected Output:** HTTP 200 responses for both course contexts with distinct system messages.
- **Actual Output:** HTTP 200 with the expected chatbot responses.
- **Result: PASSED**
- **Pytest Code:**

Python

```
paths = ['data-science', 'artificial-intelligence']

for path in paths:
    response = client.post('/v1/chat',
                           json={
                               'message': 'Test message',
                               'session_id': f'test-session-{path}',
                               'path_param': path
                           })
    assert response.status_code == 200
```

- **Captured Logs (Partial):**

Unset

```
INFO chatbot-api:run.py:50 Chatbot API endpoint called
DEBUG chatbot-api:run.py:56 Request params - path_param:
data-science
INFO chatbot-api:run.py:76 Sending request to Together API
DEBUG chatbot-api:run.py:88 Received response from API: 'Response
with context...'
INFO chatbot-api:run.py:94 Successfully processed chatbot request
```